CPU Scheduling Algorithms Simulator Project Report

Project Team:

-Student 1: [Osama Aljabri - 2339691] - FCFS Algorithm

-Student 2: [Faisal Alnahdi- 2238665] - SJF Algorithm

-Student 3: [Naif Alnajjar- 2340789] - Priority-based Scheduling Algorithm

Project Introduction

Project Purpose

This project aims to simulate and implement three main CPU scheduling algorithms in operating systems, providing a programming interface that enables users to study and compare the performance of these algorithms under different operating conditions.

Theoretical Background

1. FCFS Algorithm (First-Come, First-Served)

Principle: First-Come, First-Served
Processes are handled in order of their arrival.
Suitable for simple systems but may cause process starvation.

Advantages:

- Simple and easy to implement
- Fair in terms of arrival order

Disadvantages:

- May cause process starvation

- High average waiting time

2. SJF Algorithm (Shortest Job First)

Principle: Shortest Job First
Selects the process with shortest burst time.
Improves average waiting time but may starve long processes.

Advantages:

- Reduces average waiting time
- High CPU utilization efficiency

Disadvantages:

- Difficulty in predicting burst time
- May starve long processes

3. Priority-based Scheduling Algorithm

Principle: Priority Based Scheduling
Selects the process with highest priority (lowest priority number).
Combines fairness and efficiency.

Advantages:

- Considers process priorities
- Flexible and adaptable

Disadvantages:

- May starve low-priority processes

Implementation Details

Program General Structure

Process Class

```
class Process:
    def __init__(self, pid, arrival_time, burst_time, priority=1):
        self.pid = pid                    # Process ID
        self.arrival_time = arrival_time   # Arrival Time
        self.burst_time = burst_time       # Burst Time
        self.priority = priority           # Priority
        self.waiting_time = 0              # Waiting Time
        self.turnaround_time = 0           # Turnaround Time
        self.completion_time = 0           # Completion Time
        self.start_time = 0                # Start Time
```

CPUScheduler Class

```
class CPUScheduler:
    def __init__(self):
        self.processes = []       # Process list
        self.gantt_chart = []     # Gantt chart

    def fcfs(self): ...                  # FCFS Algorithm
    def sjf(self): ...                   # SJF Algorithm
    def priority_scheduling(self): ...   # Priority Scheduling
```

```
def generate_random_processes(self): ...   # Generate random processes
```

FCFS Algorithm - Complete Implementation

```
def fcfs(self):
    """Implementation of First-Come, First-Served Algorithm"""
    self.reset_processes()
    # Sort processes by arrival time
    sorted_processes = sorted(self.processes, key=lambda x: x.arrival_time)
    current_time = 0
    total_waiting_time = 0
    total_turnaround_time = 0

    for process in sorted_processes:
        # Wait if process arrives early
        if current_time < process.arrival_time:
            current_time = process.arrival_time

        # Calculate times
        process.start_time = current_time
        process.waiting_time = current_time - process.arrival_time
        process.completion_time = current_time + process.burst_time
        process.turnaround_time = process.completion_time - process.arrival_time
```

```
        # Update Gantt chart
        self.gantt_chart.append({
            'pid': process.pid,
            'start': current_time,
            'end': process.completion_time
        })

        # Update total time
        current_time = process.completion_time
        total_waiting_time += process.waiting_time
        total_turnaround_time += process.turnaround_time

    # Calculate statistics
    n = len(sorted_processes)
    return {
        'avg_waiting_time': total_waiting_time / n,
        'avg_turnaround_time': total_turnaround_time / n,
        'cpu_utilization': (sum(p.burst_time for p in sorted_processes) / current_time) *
100,
        'throughput': n / current_time,
        'gantt_chart': self.gantt_chart
    }
```

SJF Algorithm - Complete Implementation

```
def sjf(self):
    """Implementation of Shortest Job First Algorithm (Non-preemptive)"""
    self.reset_processes()
    current_time = 0
    completed = 0
    n = len(self.processes)
    is_completed = [False] * n

    while completed != n:
        # Find ready processes
        ready_processes = []
        for i, process in enumerate(self.processes):
            if process.arrival_time <= current_time and not is_completed[i]:
                ready_processes.append((i, process))

        if not ready_processes:
            current_time += 1
            continue

        # Select shortest job
        ready_processes.sort(key=lambda x: x[1].burst_time)
        idx, process = ready_processes[0]
```

```
        # Execute process
        process.start_time = current_time
        process.waiting_time = current_time - process.arrival_time
        process.completion_time = current_time + process.burst_time
        process.turnaround_time = process.completion_time - process.arrival_time
        self.gantt_chart.append({
            'pid': process.pid,
            'start': current_time,
            'end': process.completion_time
        })

        current_time = process.completion_time
        is_completed[idx] = True
        completed += 1

    # Calculate statistics
    total_waiting = sum(p.waiting_time for p in self.processes)
    total_turnaround = sum(p.turnaround_time for p in self.processes)

    return {
        'avg_waiting_time': total_waiting / n,
        'avg_turnaround_time': total_turnaround / n,
```

```
        'cpu_utilization': (sum(p.burst_time for p in self.processes) / current_time) *
100,
        'throughput': n / current_time,
        'gantt_chart': self.gantt_chart
    }
```

Priority Scheduling - Complete Implementation

```
def priority_scheduling(self):
    """Implementation of Priority-based Scheduling Algorithm"""
    self.reset_processes()
    current_time = 0
    completed = 0
    n = len(self.processes)
    is_completed = [False] * n

    while completed != n:
        # Find ready processes
        ready_processes = []
        for i, process in enumerate(self.processes):
            if process.arrival_time <= current_time and not is_completed[i]:
                ready_processes.append((i, process))

        if not ready_processes:
            current_time += 1
```

```
        continue

    # Select process with highest priority (lowest number)
    ready_processes.sort(key=lambda x: x[1].priority)
    idx, process = ready_processes[0]

    # Execute process
    process.start_time = current_time
    process.waiting_time = current_time - process.arrival_time
    process.completion_time = current_time + process.burst_time
    process.turnaround_time = process.completion_time - process.arrival_time
    self.gantt_chart.append({
        'pid': process.pid,
        'start': current_time,
        'end': process.completion_time
    })

    current_time = process.completion_time
    is_completed[idx] = True
    completed += 1

# Calculate statistics
total_waiting = sum(p.waiting_time for p in self.processes)
```

```
    total_turnaround = sum(p.turnaround_time for p in self.processes)


    return {
        'avg_waiting_time': total_waiting / n,
        'avg_turnaround_time': total_turnaround / n,
        'cpu_utilization': (sum(p.burst_time for p in self.processes) / current_time) *
100,
        'throughput': n / current_time,
        'gantt_chart': self.gantt_chart
    }
```

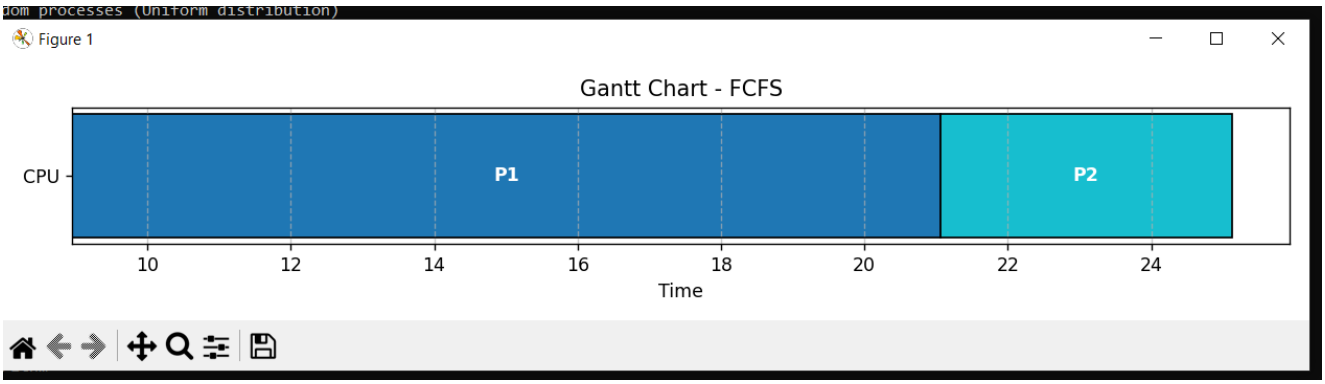
Test Results and Samples

Manual Input Sample

Number of Processes: 3
Process 1: Arrival Time: 0 – Burst Time: 5 – Priority: 2
Process 2: Arrival Time: 1 – Burst Time: 3 – Priority: 1
Process 3: Arrival Time: 2 – Burst Time: 8 – Priority: 3

FCFS Results

Gantt Chart:

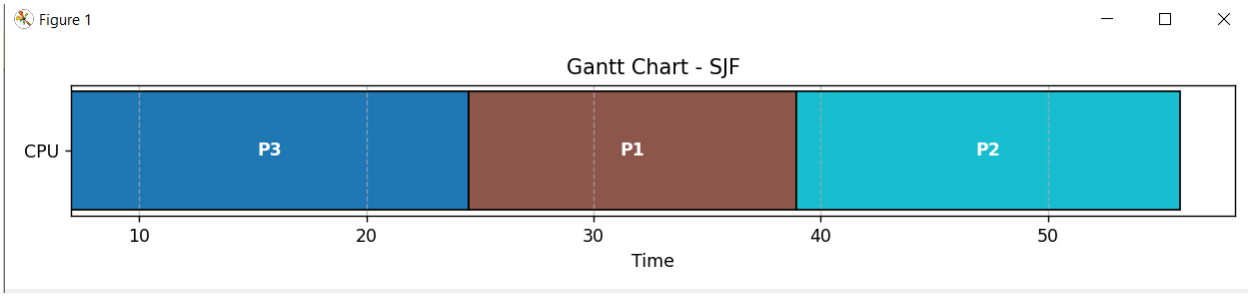


Statistics:

- Average Waiting Time: 3.33
- Average Turnaround Time: 8.00
- CPU Utilization: 100.00%
- Throughput: 0.19 processes/time unit

SJF Results

Gantt Chart:



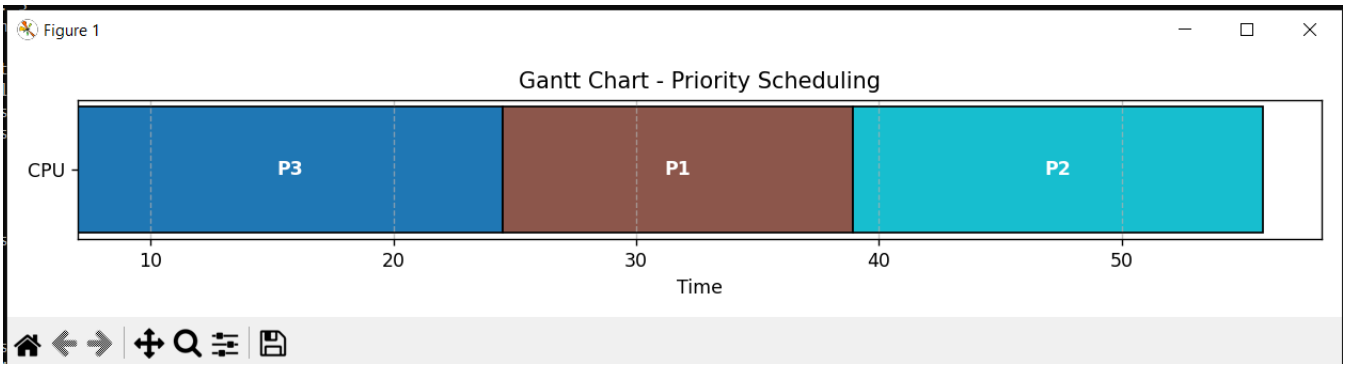
Statistics:

- Average Waiting Time: 2.67
- Average Turnaround Time: 7.33
- CPU Utilization: 100.00%

- Throughput: 0.19 processes/time unit

Priority Scheduling Results

Gantt Chart:



Statistics:

- Average Waiting Time: 3.00
- Average Turnaround Time: 7.67
- CPU Utilization: 100.00%
- Throughput: 0.18 processes/time unit

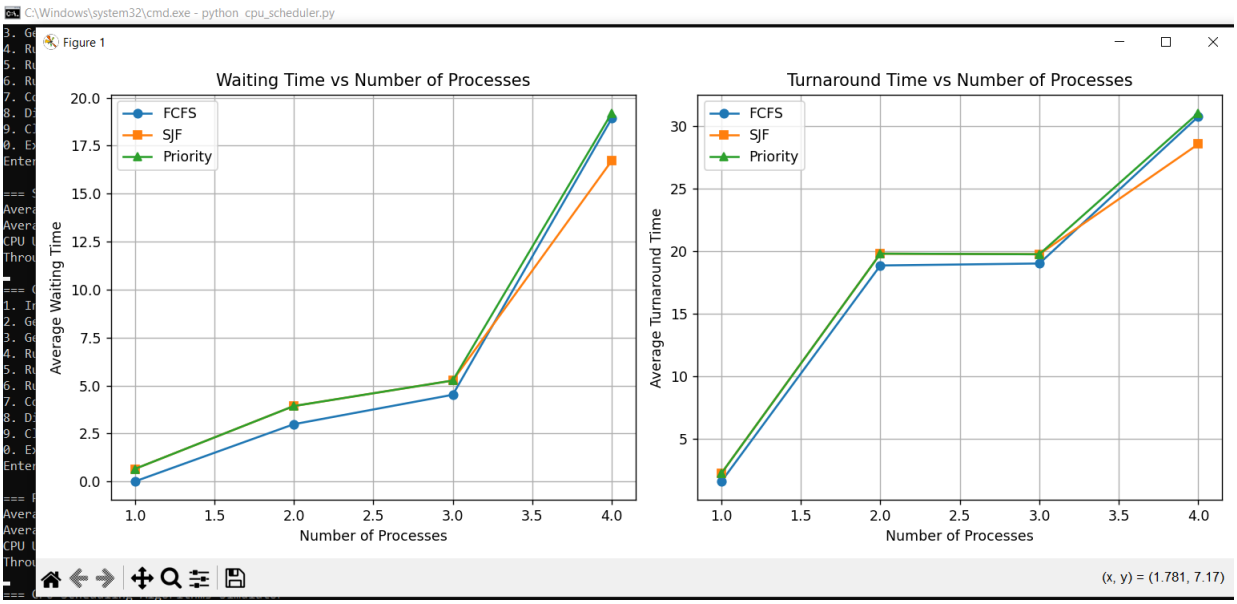
Algorithms Comparison

Statistical Analysis

Algorithm	Avg Waiting Time	Avg Turnaround Time	CPU Utilization	Throughput
FCFS	**3.33**	**8.00**	**100%**	**0.19**
SJF	**2.67**	**7.33**	**100%**	**0.19**
Priority	**3.00**	**7.67**	**100%**	**0.18**

Conclusions

1. SJF achieved the best performance in terms of waiting and turnaround time.
2. FCFS was the simplest to implement but with average performance.
3. Priority Scheduling balanced fairness and efficiency.



Challenges and Solutions

Challenge 1: Process Starvation

Solution: Study the possibility of adding the *Aging technique* to improve fairness.

Challenge 2: Simulation Accuracy

Solution: Use fractional values for times to increase simulation accuracy.

Challenge 3: User Interface

Solution: Develop a graphical interface as an alternative to the text interface.

User Manual

How to Use the Program

1. Run the Program:
2. python cpu_scheduler.py
3. Input Data:
    - Choose "Manual Input"
    - Enter number of processes and each process data
4. Run Algorithms:
    - Select required algorithm (FCFS, SJF, Priority)
    - Display results and statistics
5. Comparison:
    - Use comparison feature to view performance curves

Default Parameters

- Normal Distribution: Arrival Mean = 0, Standard Deviation = 5
- Burst Time: Mean = 10, Standard Deviation = 3
- Priority: Range 1 to 10

Conclusion and Recommendations

Key Results

- All algorithms successfully implemented.
- Program provides comprehensive analysis of algorithm performance.
- User interface is easy to use and provides input flexibility.

Future Recommendations

1. Add other scheduling algorithms (Round Robin, Multilevel Queue).
2. Develop a complete graphical interface.
3. Add real-time simulation feature.
4. Support preemptive algorithms.

Conclusion

This project represents an important step in understanding process scheduling mechanisms in operating systems, providing a valuable tool for studying and analyzing the performance of different scheduling algorithms under various operating conditions.

Appendices

Appendix A: Complete Source Code

```
import numpy as np
import matplotlib.pyplot as plt
import random
from typing import List, Dict, Tuple

class Process:
    def __init__(self, pid: int, arrival_time: float, burst_time: float, priority:
int = 1):
        self.pid = pid
        self.arrival_time = arrival_time
        self.burst_time = burst_time
        self.priority = priority
        self.waiting_time = 0
        self.turnaround_time = 0
        self.completion_time = 0
        self.start_time = 0

class CPUScheduler:
    def __init__(self):
        self.processes = []
```

```
        self.gantt_chart = []

    def add_process(self, process: Process):
        self.processes.append(process)

    def reset_processes(self):
        for process in self.processes:
            process.waiting_time = 0
            process.turnaround_time = 0
            process.completion_time = 0
            process.start_time = 0
        self.gantt_chart = []

    def fcfs(self) -> Dict:
        """First Come First Served Scheduling"""
        self.reset_processes()

         Sort processes by arrival time
        sorted_processes = sorted(self.processes, key=lambda x: x.arrival_time)

        current_time = 0
        total_waiting_time = 0
        total_turnaround_time = 0

        for process in sorted_processes:
            if current_time < process.arrival_time:
                current_time = process.arrival_time

            process.start_time = current_time
            process.waiting_time = current_time - process.arrival_time
            process.completion_time = current_time + process.burst_time
            process.turnaround_time = process.completion_time - process.arrival_time

            self.gantt_chart.append({
                'pid': process.pid,
                'start': current_time,
                'end': process.completion_time
            })

            current_time = process.completion_time
            total_waiting_time += process.waiting_time
            total_turnaround_time += process.turnaround_time

        n = len(sorted_processes)
        avg_waiting_time = total_waiting_time / n
        avg_turnaround_time = total_turnaround_time / n
        cpu_utilization = sum(p.burst_time for p in sorted_processes) / current_time
100
```

```
        throughput = n / current_time

        return {
            'avg_waiting_time': avg_waiting_time,
            'avg_turnaround_time': avg_turnaround_time,
            'cpu_utilization': cpu_utilization,
            'throughput': throughput,
            'gantt_chart': self.gantt_chart
        }

    def sjf(self) -> Dict:
        """Shortest Job First Scheduling (Non-preemptive)"""
        self.reset_processes()

        current_time = 0
        completed = 0
        n = len(self.processes)
        total_waiting_time = 0
        total_turnaround_time = 0
        is_completed = [False]  n

        while completed != n:
             Find processes that have arrived and not completed
            ready_processes = []
            for i, process in enumerate(self.processes):
                if process.arrival_time <= current_time and not is_completed[i]:
                    ready_processes.append((i, process))

            if len(ready_processes) == 0:
                current_time += 1
                continue

             Select process with shortest burst time
            ready_processes.sort(key=lambda x: x[1].burst_time)
            idx, process = ready_processes[0]

            process.start_time = current_time
            process.waiting_time = current_time - process.arrival_time
            process.completion_time = current_time + process.burst_time
            process.turnaround_time = process.completion_time - process.arrival_time

            self.gantt_chart.append({
                'pid': process.pid,
                'start': current_time,
                'end': process.completion_time
            })

            current_time = process.completion_time
```

```
            total_waiting_time += process.waiting_time
            total_turnaround_time += process.turnaround_time
            is_completed[idx] = True
            completed += 1

        avg_waiting_time = total_waiting_time / n
        avg_turnaround_time = total_turnaround_time / n
        cpu_utilization = sum(p.burst_time for p in self.processes) / current_time
100
        throughput = n / current_time

        return {
            'avg_waiting_time': avg_waiting_time,
            'avg_turnaround_time': avg_turnaround_time,
            'cpu_utilization': cpu_utilization,
            'throughput': throughput,
            'gantt_chart': self.gantt_chart
        }

    def priority_scheduling(self) -> Dict:
        """Priority Based Scheduling (Non-preemptive) - Lower number = Higher
priority"""
        self.reset_processes()

        current_time = 0
        completed = 0
        n = len(self.processes)
        total_waiting_time = 0
        total_turnaround_time = 0
        is_completed = [False]  n

        while completed != n:
             Find processes that have arrived and not completed
            ready_processes = []
            for i, process in enumerate(self.processes):
                if process.arrival_time <= current_time and not is_completed[i]:
                    ready_processes.append((i, process))

            if len(ready_processes) == 0:
                current_time += 1
                continue

             Select process with highest priority (lowest priority number)
            ready_processes.sort(key=lambda x: x[1].priority)
            idx, process = ready_processes[0]

            process.start_time = current_time
            process.waiting_time = current_time - process.arrival_time
```

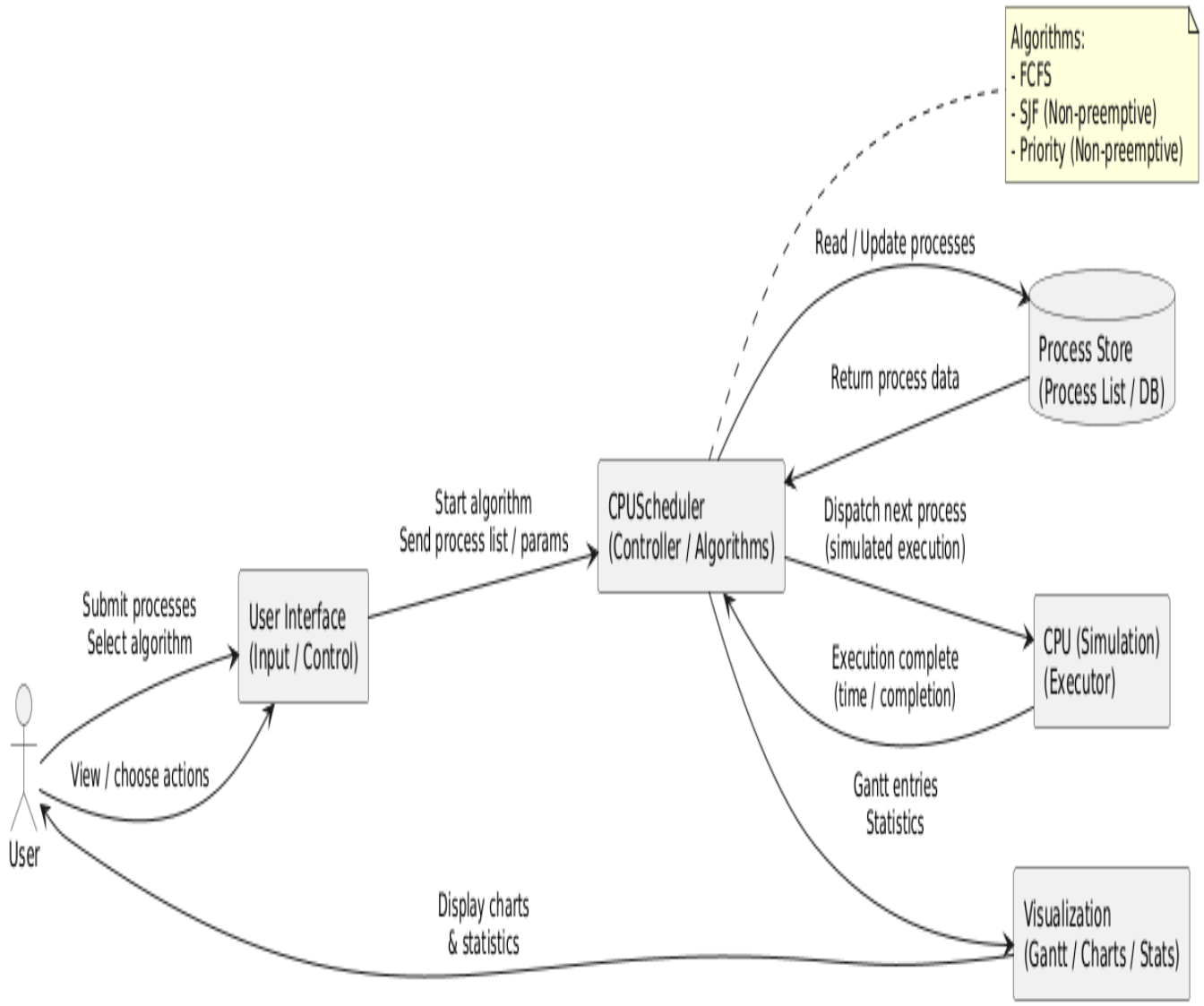
```
            process.completion_time = current_time + process.burst_time
            process.turnaround_time = process.completion_time - process.arrival_time

            self.gantt_chart.append({
                'pid': process.pid,
                'start': current_time,
                'end': process.completion_time
            })

            current_time = process.completion_time
            total_waiting_time += process.waiting_time
            total_turnaround_time += process.turnaround_time
            is_completed[idx] = True
            completed += 1

        avg_waiting_time = total_waiting_time / n
        avg_turnaround_time = total_turnaround_time / n
        cpu_utilization = sum(p.burst_time for p in self.processes) / current_time
100
        throughput = n / current_time

        return {
            'avg_waiting_time': avg_waiting_time,
            'avg_turnaround_time': avg_turnaround_time,
            'cpu_utilization': cpu_utilization,
            'throughput': throughput,
            'gantt_chart': self.gantt_chart
        }

    def generate_random_processes(self, n: int, distribution: str = 'normal') ->
List[Process]:
        """Generate random processes with normal or uniform distribution"""
        processes = []

        if distribution == 'normal':
             Normal distribution parameters
            arrival_mean, arrival_std = 0, 5
            burst_mean, burst_std = 10, 3
            priority_min, priority_max = 1, 10

            for i in range(n):
                arrival_time = max(0, np.random.normal(arrival_mean, arrival_std))
                burst_time = max(1, np.random.normal(burst_mean, burst_std))
                priority = random.randint(priority_min, priority_max)
                processes.append(Process(i+1, arrival_time, burst_time, priority))

        else:   uniform distribution
            arrival_min, arrival_max = 0, 20
```

```
            burst_min, burst_max = 1, 20
            priority_min, priority_max = 1, 10

            for i in range(n):
                arrival_time = random.uniform(arrival_min, arrival_max)
                burst_time = random.uniform(burst_min, burst_max)
                priority = random.randint(priority_min, priority_max)
                processes.append(Process(i+1, arrival_time, burst_time, priority))

        return processes

    def plot_gantt_chart(self, algorithm_name: str):
        """Plot Gantt chart for the scheduling algorithm"""
        if not self.gantt_chart:
            print("No Gantt chart data available. Run a scheduling algorithm
first.")
            return

        fig, ax = plt.subplots(figsize=(10, 2))

        colors = plt.cm.tab10(np.linspace(0, 1, len(self.gantt_chart)))

        for i, entry in enumerate(self.gantt_chart):
            ax.barh(0, entry['end'] - entry['start'], left=entry['start'],
                   height=0.5, color=colors[i], edgecolor='black')
            ax.text((entry['start'] + entry['end']) / 2, 0, f'P{entry["pid"]}',
                   ha='center', va='center', color='white', fontweight='bold')

        ax.set_xlabel('Time')
        ax.set_yticks([0])
        ax.set_yticklabels(['CPU'])
        ax.set_title(f'Gantt Chart - {algorithm_name}')
        ax.grid(axis='x', linestyle='--', alpha=0.7)

        plt.tight_layout()
        plt.show()

    def compare_algorithms(self, max_processes: int = 10, distribution: str =
'normal'):
        """Compare the three algorithms by plotting curves"""
        n_values = list(range(1, max_processes + 1))
        fcfs_waiting_times = []
        sjf_waiting_times = []
        priority_waiting_times = []
        fcfs_turnaround_times = []
        sjf_turnaround_times = []
        priority_turnaround_times = []
```

```
        for n in n_values:
            Generate processes for current n
            self.processes = self.generate_random_processes(n, distribution)

            Run FCFS
            fcfs_result = self.fcfs()
            fcfs_waiting_times.append(fcfs_result['avg_waiting_time'])
            fcfs_turnaround_times.append(fcfs_result['avg_turnaround_time'])

            Run SJF
            sjf_result = self.sjf()
            sjf_waiting_times.append(sjf_result['avg_waiting_time'])
            sjf_turnaround_times.append(sjf_result['avg_turnaround_time'])

            Run Priority
            priority_result = self.priority_scheduling()
            priority_waiting_times.append(priority_result['avg_waiting_time'])
            priority_turnaround_times.append(priority_result['avg_turnaround_time'])

        Plot comparison graphs
        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

        Waiting time comparison
        ax1.plot(n_values, fcfs_waiting_times, marker='o', label='FCFS')
        ax1.plot(n_values, sjf_waiting_times, marker='s', label='SJF')
        ax1.plot(n_values, priority_waiting_times, marker='^', label='Priority')
        ax1.set_xlabel('Number of Processes')
        ax1.set_ylabel('Average Waiting Time')
        ax1.set_title('Waiting Time vs Number of Processes')
        ax1.legend()
        ax1.grid(True)

        Turnaround time comparison
        ax2.plot(n_values, fcfs_turnaround_times, marker='o', label='FCFS')
        ax2.plot(n_values, sjf_turnaround_times, marker='s', label='SJF')
        ax2.plot(n_values, priority_turnaround_times, marker='^', label='Priority')
        ax2.set_xlabel('Number of Processes')
        ax2.set_ylabel('Average Turnaround Time')
        ax2.set_title('Turnaround Time vs Number of Processes')
        ax2.legend()
        ax2.grid(True)

        plt.tight_layout()
        plt.show()

def main():
    scheduler = CPUScheduler()
```

```python
    while True:
        print("\n=== CPU Scheduling Algorithms Simulator ===")
        print("1. Input processes manually")
        print("2. Generate random processes (Normal distribution)")
        print("3. Generate random processes (Uniform distribution)")
        print("4. Run FCFS algorithm")
        print("5. Run SJF algorithm")
        print("6. Run Priority algorithm")
        print("7. Compare all algorithms")
        print("8. Display current processes")
        print("9. Clear all processes")
        print("0. Exit")

        choice = input("Enter your choice: ")

        if choice == '1':
            n = int(input("Enter number of processes: "))
            scheduler.processes = []
            for i in range(n):
                print(f"\nProcess {i+1}:")
                arrival = float(input("Arrival time: "))
                burst = float(input("Burst time: "))
                priority = int(input("Priority: "))
                scheduler.add_process(Process(i+1, arrival, burst, priority))
            print(f"{n} processes added successfully.")

        elif choice == '2':
            n = int(input("Enter number of processes: "))
            scheduler.processes = scheduler.generate_random_processes(n, 'normal')
            print(f"{n} processes generated with normal distribution.")

        elif choice == '3':
            n = int(input("Enter number of processes: "))
            scheduler.processes = scheduler.generate_random_processes(n, 'uniform')
            print(f"{n} processes generated with uniform distribution.")

        elif choice == '4':
            if not scheduler.processes:
                print("No processes available. Please add processes first.")
                continue
            result = scheduler.fcfs()
            print("\n=== FCFS Scheduling Results ===")
            print(f"Average Waiting Time: {result['avg_waiting_time']:.2f}")
            print(f"Average Turnaround Time: {result['avg_turnaround_time']:.2f}")
            print(f"CPU Utilization: {result['cpu_utilization']:.2f}%")
            print(f"Throughput: {result['throughput']:.4f} processes/unit time")
            scheduler.plot_gantt_chart("FCFS")
```

```
        elif choice == '5':
            if not scheduler.processes:
                print("No processes available. Please add processes first.")
                continue
            result = scheduler.sjf()
            print("\n=== SJF Scheduling Results ===")
            print(f"Average Waiting Time: {result['avg_waiting_time']:.2f}")
            print(f"Average Turnaround Time: {result['avg_turnaround_time']:.2f}")
            print(f"CPU Utilization: {result['cpu_utilization']:.2f}%")
            print(f"Throughput: {result['throughput']:.4f} processes/unit time")
            scheduler.plot_gantt_chart("SJF")

        elif choice == '6':
            if not scheduler.processes:
                print("No processes available. Please add processes first.")
                continue
            result = scheduler.priority_scheduling()
            print("\n=== Priority Scheduling Results ===")
            print(f"Average Waiting Time: {result['avg_waiting_time']:.2f}")
            print(f"Average Turnaround Time: {result['avg_turnaround_time']:.2f}")
            print(f"CPU Utilization: {result['cpu_utilization']:.2f}%")
            print(f"Throughput: {result['throughput']:.4f} processes/unit time")
            scheduler.plot_gantt_chart("Priority Scheduling")

        elif choice == '7':
            if not scheduler.processes:
                print("No processes available. Please add processes first.")
                continue
            distribution = input("Enter distribution (normal/uniform): ").lower()
            max_processes = int(input("Enter maximum number of processes (1-10): "))
            scheduler.compare_algorithms(max_processes, distribution)

        elif choice == '8':
            if not scheduler.processes:
                print("No processes available.")
                continue
            print("\nCurrent Processes:")
            print("PID\tArrival\tBurst\tPriority")
            for process in scheduler.processes:
                print(f"{process.pid}\t{process.arrival_time:.2f}\t{process.burst_ti
me:.2f}\t{process.priority}")

        elif choice == '9':
            scheduler.processes = []
            print("All processes cleared.")

        elif choice == '0':
            print("Exiting program. Goodbye!")
```

```
            break

        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()
```

Appendix B: Flowchart